计算概论 A 期中复习讲义

题目目录

编号	题目	主要内容
01	阶梯电价购电	分段计算、浮点输出
02	记数问题	枚举、数位拆分
03	小于当前数的数	双重循环、数组计数
04	门诊计数	排序、时间模拟
05	除法计算	余数状态、循环节
06	Vika的贺卡	二维数组、按列匹配
07	判断是否为C语言的合法标识符	整行读取、字符判断
08	位置查找	字符映射、首次位置
09	DNA排序（A）	逆序对、稳定排序
10	重排空格	字符串扫描、空格分配
11	流感	分层传播、数组模拟队列
12	柱状图上的最大矩形	单调栈
13	照亮房间	可视核与方向约束
14	炸鸡排	贪心与可行性
15	愉悦的旋律	贪心分段、出现标记
16	啤酒厂选址	环形距离、枚举选址

01　阶梯电价购电

考点： 分段计算、浮点输出

一、题目

为了提高能源效率，北京市对居民用电实行阶梯式递增电价，第一档为0~240度（含），每度电为0.4883元，第二档为241~400度（含），每度电为0.5383元；第三档为400度以上，每度电为0.7883元。阶梯电价的含义是，如果本月用电量是246度，那么其中240度执行第一档的价格0.4883×240，超过240度的部分还有6度，执行第二档价格0.5383×6，实际电费为两档电费的总和。

输入为一个整数，为持有的人民币m，以元为单位。若这m元全部用来购电，求本月可购买的用电度数d，保留到小数点后两位。

输入格式

输入为一个整数，为持有的人民币m，以元为单位。

输出格式

本月可购买的用电度数d，保留到小数点后两位。

样例输入

```
100
```

样例输出

```
204.79
```

原题提示

以double类型定义可购买电的度数. setprecision 需 include iomanip

二、解析

题意与范围

输入整数金额m，输出按题目给定阶梯价格能购买的度数，保留两位小数。

思路

先算累计费用：$C_1=240\times0.4883=117.192$；$C_2=C_1+160\times0.5383=203.32$。金额不超过$C_1$时$d=m/0.4883$；不超过$C_2$时$d=240+(m-C_1)/0.5383$；否则$d=400+(m-C_2)/0.7883$。

复杂度

O(1)时间，O(1)额外空间。

三、参考代码（C++11）

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double m;
    cin >> m;
    const double p1 = 0.4883, p2 = 0.5383, p3 = 0.7883;
    const double cost1 = 240 * p1;
    const double cost2 = cost1 + 160 * p2;
    double d;
```

```
    // 输入是钱，因此用“累计费用”判断属于哪一档
    if (m <= cost1) d = m / p1;
    else if (m <= cost2) d = 240 + (m - cost1) / p2;
    else d = 400 + (m - cost2) / p3;
    cout << fixed << setprecision(2) << d << '\n'; // 注意输出的格式
    return 0;
}
```

02　记数问题

考点： 枚举、数位拆分

一、题目

试计算在区间 1 到 n 的所有整数中，数字 x(0 ≤ x ≤ 9)共出现了多少次？例如，在 1 到 11 中，即在 1、2、3、4、5、6、7、8、9、10、11 中，数字 1 出现了 4 次，数字2出现了1次。

输入格式

输入共 1 行，包含 2 个整数 n、x，之间用一个空格隔开。

输出格式

输出共 1 行，包含一个整数，表示 x 出现的次数。

样例输入

```
11 1
```

样例输出

```
4
```

原题提示

对于所有的数据，1≤ n ≤ 10000，0≤ x ≤ 9。

二、解析

思路1

枚举每个整数i，令t=i。反复检查t%10是不是x，再令t/=10，直到t为0。每次遇到目标数位都加1。

思路2

利用to_string函数将int转化为string来处理

复杂度

O(nlogn)时间，O(1)额外空间；本题至多检查约5万位。

三、参考代码（C++11）

```cpp
#include <iostream>
using namespace std;

int main() {
    int n, x;
    cin >> n >> x;
    int answer = 0;
    for (int i = 1; i <= n; ++i) {
        int t = i;  // 常见易错点：拆副本，不能改变控制外层循环的 i
        while (t > 0) {
            if (t % 10 == x) ++answer;
            t /= 10;
        }
    }
    // 区间从 1 开始，没有整数 0，也不补前导零
    cout << answer << '\n';
    return 0;
}
```

03　小于当前数的数

考点： 双重循环、数组计数

一、题目

给你一个数组 nums，对于其中每个元素 nums[i]，请你统计数组中比它小的所有数字的数目。

换而言之，对于每个 nums[i] 你必须计算出有效的 j 的数量，其中 j 满足 j != i 且 nums[j] < nums[i]。

输入格式

第一行是一个整数 n，表示数组大小。 接下来 n 行，每行表示数组中的一个数。

0 <= n <= 1000, 每个数都在int可以表示的范围内.

输出格式

n 行，每行代表数组中比它小的数的个数。

如果没有要输出的数据，输出"None"。

样例输入

```
5
8
1
2
2
3
```

样例输出

```
4
0
1
1
3
```

二、解析

题意与范围

n≤1000，逐个输出数组中严格小于该元素的元素个数；n=0输出None。

易错点

不是不同数值的个数；比较符号不是<=；输出须对应原输入顺序，不要先排序后丢失对应关系。

复杂度

$O(n^2)$时间，O(n)存储数组；最多约100万次比较，适合题面规模。

思考

如果n=1000000呢

三、参考代码（C++11）

```cpp
#include <iostream>
using namespace std;

int main() {
    int n, a[1000];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    if (n == 0) {
        cout << "None\n";
        return 0;
    }
```

```
    for (int i = 0; i < n; ++i) {
        int count = 0;  // 每个元素都要重新计数
        for (int j = 0; j < n; ++j) {
            // 严格小于自动排除了自身；相同数值的不同位置分别计数
            if (a[j] < a[i]) ++count;
        }
        cout << count << '\n';
    }
    return 0;
}
```

04　门诊计数

考点： 排序、时间模拟

一、题目

医院某科门诊早上开两个小时。从8点开始可以排队，9点开始正式门诊，每个病人需要看10分钟（若最后剩余不足10分钟，则不进行门诊），11点门诊结束。输入包含 n (0 <= n <= 20) 个病人的到达时间（距离8点的分钟数），输出最终得到门诊的病人数。

输入格式

第一行为病人数n。 之后有n行，每行为病人到达的时间

输出格式

一个整数：能得到门诊的病人数

样例输入

```
10
5
10
160
180
20
110
145
70
100
200
```

样例输出

```
8
```

二、解析

题意与范围

n≤20，输入到达时间（距8点分钟数），9点至11点服务，每人10分钟，输出可服务人数。

思路

先把到达时间升序排序。维护freeTime，初始60。每人实际开始时刻为max(freeTime,arrival)；若start+10≤180，则接诊并更新freeTime，否则结束。

为什么正确

按到达顺序处理能依次模拟排队；医生空闲与病人到达两个条件都满足才能开始。每次尽早开始不会减少后续可用时间。排序后，当前病人已无法完成时，后面的也不可能完成。

板演样例

原样例排序后为5,10,20,70,100,110,145,160,180,200；对应完成时间70,80,90,100,110,120,155,170，后两人不能完成，结果8。

复杂度

排序$O(n^2)$，模拟O(n)，空间O(n)；n≤20，无需复杂调度结构。

三、参考代码（C++11）

```cpp
#include <iostream>
using namespace std;

int main() {
    int n, a[20];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    // 输入没有保证有序，先用基础冒泡排序按到达时间升序排列
    for (int end = n - 1; end > 0; --end)
        for (int j = 0; j < end; ++j)
            if (a[j] > a[j + 1]) {
                int temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
            }
    int freeTime = 60, answer = 0;  // 距离 8 点的分钟数：9 点是 60
    for (int i = 0; i < n; ++i) {
        int start = freeTime;
        if (a[i] > start) start = a[i];  // 医生空闲、病人到达，缺一不可
        if (start + 10 > 180) break;    // 11 点前必须完成，恰好完成可以
        freeTime = start + 10;
        ++answer;
    }
    cout << answer << '\n';
```

```
    return 0;
}
```

05　除法计算

考点： 余数状态、循环节

一、题目

小P正在计算一个正整数除法a/b，但是他并不想知道结果具体是多少，他只想知道将结果化为小数之后，形成的循环节长度是多少。例如:1 / 90的结果为0.011111...，循环节长度为1。如果结果不为循环小数，则认为循环节长度是0。

输入格式

两个正整数a,b,满足1≤a,b≤1000

输出格式

输出a/b循环节长度

样例输入

```
2 7
```

样例输出

```
6
```

二、解析

题意与范围

1≤a,b≤1000，求a/b小数部分的最短循环节长度；有限小数输出0。

思路

整数部分不影响答案，从r=a%b开始。first[r]保存该余数第一次出现的位置，初值-1。每次先登记当前位置，再计算(r*10)%b并推进位置。遇0输出0，遇旧余数输出当前位置减首次位置。

为什么正确

固定除数时，当前余数唯一决定下一位小数和下一余数，所以重复余数之后的过程完全重复。余数只有0到b−1，故过程必定终止或重复。记录首次位置能排除循环前的不循环部分。

另外，不必担心while会一直进行下去而timeout，循环的充要条件是余数的重复，而余数最多有1000个，因此while循环不会超过一千次。

板演样例

1/6：位置0余数1，位置1余数4，位置2再次余数4，循环长2−1=1。1/8余数1→2→4→0，答案0。2/7余数2→6→4→5→1→3→2，答案6。

易错点

相同小数数字不一定意味后续重复；遇0必须按有限小数处理；初始位置0合法，未访问用-1；首次位置不得被覆盖。不要用double寻找循环节。

复杂度

O(b)时间；数组按题目上限分配1001个元素，使用的状态空间O(b)。

三、参考代码（C++11）

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    int first[1001];
    for (int i = 0; i <= 1000; ++i) first[i] = -1;
    int r = a % b, pos = 0;
    // 在产生下一位小数之前，记录余数出现的位置
    while (r != 0 && first[r] == -1) {
        first[r] = pos;
        r = (r * 10) % b;
        ++pos;
    }
    if (r == 0) cout << 0 << '\n';  // 有限小数没有循环节
    else cout << pos - first[r] << '\n';
    return 0;
}
```

06　Vika的贺卡

考点： 二维数组、贪心

一、题目

今天是Vika的生日，Vika的好朋友Arina送给了Vika一张贺卡，贺卡可以用n行m列的小写字母表示。 如果Vika能从贺卡中选出4个不同的列（4个列可以不连续），且这4列从左到右的第一列包含字母v，第二列包含字母i，第三列包含字母k，第四列包含字母a，那么Vika就会感到很惊喜。 请问，Vika是否会感到惊喜呢?

输入格式

第一行一个整数t(1<=t<=100)，代表数据的组数。 接下来t组数据： 每组数据的第一行包含两个整数n,m(1<=n,m<=20)，用空格分隔，代表贺卡的大小。 接下来n行，每行包含m个小写字母，表示贺卡的内容。

输出格式

对于每组数据，如果Vika会感到惊喜，则输出YES；否则，输出NO。

样例输入

```
5
1 4
vika
3 3
bad
car
pet
4 4
vvvv
iiii
kkkk
aaaa
4 4
vkak
iiai
avvk
viaa
4 7
vbickda
vbickda
vbickda
vbickda
```

样例输出

```
YES
NO
YES
NO
YES
```

二、解析

题意与范围

t组n×m字母网格，能否选出从左到右四个不同列，分别包含v、i、k、a？n,m≤20。

思路

维护matched表示已匹配target="vika"的前几个字母。外层按列从左往右，内层查该列是否含当前需要的字母。找到就matched++并break，进入下一列。

为什么正确

在当前可用列中尽早匹配目标字母，只会给后面的目标留下更多列：若某个解在更晚列选该字母，用当前更早列替换仍不影响后续列。因此一次从左到右扫描即可，无须回溯。

板演样例

一行vbickda：选第1、3、5、7列。四行一列v/i/k/a虽含四种字母，仍只能选一列，答案NO。

易错点

不同字母允许来自不同行；列可以不相邻，但必须严格递增。找到后不break可能在同一列匹配多个字母。matched达到4后不能访问target[4]。

复杂度

每组O(nm)时间，O(nm)存储网格。

三、参考代码（C++11）

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int t;
    cin >> t;
    while (t--) {
        int n, m;
        cin >> n >> m;
        string grid[20];
        for (int r = 0; r < n; ++r) cin >> grid[r];
        string target = "vika";
        int matched = 0;
        for (int c = 0; c < m && matched < 4; ++c) {
            for (int r = 0; r < n; ++r) {
                if (grid[r][c] == target[matched]) {
                    ++matched;
                    break;  // 一列最多匹配一个字母，不能重复使用该列
                }
            }
        }
        cout << (matched == 4 ? "YES" : "NO") << '\n';
    }
```

```
    return 0;
}
```

07　判断是否为C语言的合法标识符

考点： 整行读取、字符判断

一、题目

C语言中的合法标识符的定义为：以下划线或字母开头的字母数字串（含下划线）。 完成一个程序实现对输入的n个字符串进行判定，是否为C语言的合法标识符。如果是则输出1，不是则输出0

输入格式

输入的第一行为一个数字，表明有几个输入字串。 后面每一行为一个长度不超过80的字符串。

输出格式

对所有输入的字符串进行判断，是合法标识符则输出1，回车。否则输出0，回车。

样例输入

```
5
hello_world
my god
i
_stdio
008A
```

样例输出

```
1
0
1
1
0
```

三、参考代码（C++11）

```
#include <iostream>
#include <string>
#include <limits>
using namespace std;

int main() {
```

```
    int n;
    cin >> n;
    cin.ignore(); // 注意此操作非常重要
    // 或者char c; cin.get(c);用c“吃掉”遗留的换行符
    while (n--) {
        string s;
        getline(cin, s);  // 必须保留行内空格，my god 是一个待判断字符串
        bool ok = !s.empty();
        for (int i = 0; i < static_cast<int>(s.size()); ++i) {
            char c = s[i];
            bool letter = ('a' <= c && c <= 'z') || ('A' <= c && c <= 'Z');
            bool digit = ('0' <= c && c <= '9');
            if (!(letter || c == '_' || (i > 0 && digit))) {
                ok = false;
                break;
            }
        }
        // 只检查题面给定的字符规则，不额外检查 C 语言关键字
        cout << (ok ? 1 : 0) << '\n';
    }
    return 0;
}
```

08　位置查找

考点： 字符映射、首次位置

一、题目

编写一个程序，用于查找并输出在给定的由英文组成的字符串中，每个不重复字符首次出现的位置。注意，不区分大小写（即字符 'p' 和 'P' 均视作相同字符）。

输入格式

输入一个连续字符串（字符串内无空格），长度不超过200个字符。字符串可以包含小写和大写字母以及其他字符。

输出格式

对于字符串中的每一个不重复的英文字符（不区分大小写），输出该字符（以小写形式）及其首次出现的索引位置，并用空格间隔开。每个字符及其位置应该在新的一行中输出。输出请按照字典序输出字母及其位置，未出现的字母不要输出。

样例输入

```
Programming!@pku_cs
```

样例输出

```
a 5
c 17
g 3
i 8
k 14
m 6
n 9
o 2
p 0
r 1
s 18
u 15
```

原题提示

·测试数据中可能有其他非英文字符如数字、句号、叹号等，请注意判断，且这些字符也会占据索引位置。例如，在输入字符串 "P1!2kKu" 中，字符 'k' 的首次出现位置为 4。 ·字符不区分大小写。 ·字符串下标从0开始。

二、解析

题意与范围

无空格字符串长≤200，按字母序输出每个不区分大小写的英文字母及其首次出现的原串下标。

思路

26元素first数组全设-1。遍历原串：大写转小写，确认是字母后映射到c−'a'。仅未记录时写入i。最后按0到25顺序输出已记录项。

为什么正确

从左到右第一次写入的i一定最小，之后不覆盖保持首次位置；字母编号从a到z遍历自然保证输出字典序。

板演样例

P1!2kKu：p在0，k在4，u在6；按字母序输出k 4、p 0、u 6。数字和标点虽不输出，但仍占据原串索引。

易错点

不要先删除标点再计下标；不能用0代表未出现，因为首字符下标就是0；不能用int first[26]={-1}期待所有元素都是-1。

复杂度

O(L+26)时间，除输入串外O(26)额外空间。

三、参考代码（C++11）

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s;
    cin >> s;
    int first[26];
    for (int i = 0; i < 26; ++i) first[i] = -1;
    for (int i = 0; i < static_cast<int>(s.size()); ++i) {
        char c = s[i];
        if ('A' <= c && c <= 'Z') c = c - 'A' + 'a';
        if ('a' <= c && c <= 'z' && first[c - 'a'] == -1)
            first[c - 'a'] = i;  // 记录原串下标，非字母也占位
    }
    for (int k = 0; k < 26; ++k)
        if (first[k] != -1)
            cout << char('a' + k) << ' ' << first[k] << '\n';
    return 0;
}
```

09　DNA排序（A）

考点： 逆序对、稳定排序

一、题目

作为医学生的你，一定对DNA不陌生。DNA可以看作是由字母'A''C''G''T'组成的字符串，现在有一些DNA序列需要你排序，排序的规则是按照DNA序列中逆序对的个数。逆序对的定义就是字符串中，在字母表中靠后的字母排在了字母表中靠前的字母的前面。举例来说，字符串"DAABEC"的逆序对数为5，分别是"D-A""D-A""D-B","D-C"和"E-C"，而"AACEDGG'' 就只有"E-D"这一个逆序对，"ABCDE"就没有逆序对。现在，给你一些DNA序列，请你将它们按照逆序对的个数从小到大排序。

输入格式

第一行是两个整数n和m，0<n<=50，表示字符串的长度，0<m<=100，表示字符串的个数。 接下来是m行，每一行是一个长度为n的字符串。

输出格式

m行，将m个字符串按照逆序对数从小到大的顺序输出。如果有逆序对数相等的字符串，按照它们在输入中的相对顺序输出。

样例输入

```
9 5
ACCGTTGAC
GGCTACGTA
```

```
ATTCGACGT
TGACTGACT
CCACGTACT
```

样例输出

```
CCACGTACT
ACCGTTGAC
ATTCGACGT
TGACTGACT
GGCTACGTA
```

二、解析

易错点

等于时不构成逆序；排序分数而不是DNA字典序；不能只换分数不换串；冒泡条件用>而非>=。注意不能用sort，因为sort排序是不稳定的。

快排、堆排都是不稳定的，归并排序也是稳定的O(nlogn)的排序算法，感兴趣的同学可以自行搜索。

复杂度

计数$O(mn^2)$；排序最多$O(m^2)$次分数比较，字符串赋值按O(n)计，则交换拷贝最坏$O(m^2n)$，总$O(mn^2+m^2n)$；空间O(mn)。

三、参考代码（C++11）

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    string dna[100];
    int inversions[100] = {};
    for (int k = 0; k < m; ++k) {
        cin >> dna[k];
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                if (dna[k][i] > dna[k][j]) ++inversions[k];
    }
    // 相邻交换且仅在严格大于时交换，保证相同分数的原顺序不变（即稳定排序）
    for (int end = m - 1; end > 0; --end)
        for (int j = 0; j < end; ++j)
            if (inversions[j] > inversions[j + 1]) {
                int temp = inversions[j];
```

```
                inversions[j] = inversions[j + 1];
                inversions[j + 1] = temp;
                string tempString = dna[j];
                dna[j] = dna[j + 1];
                dna[j + 1] = tempString;  // 分数和字符串必须一起移动
            }
    for (int k = 0; k < m; ++k) cout << dna[k] << '\n';
    return 0;
}
```

10　重排空格

考点： 字符串扫描、空格分配

一、题目

给你一个字符串 text，该字符串由若干被空格包围的单词组成。每个单词由一个或者多个小写英文字母组成，并且两个单词之间至少存在一个空格。 请你重新排列空格，使每对相邻单词之间的空格数目都 相等 ，并尽可能 最大化 该数目。如果不能重新平均分配所有空格，请 将多余的空格放置在字符串末尾 ，这也意味着输出的字符串应当与原字符串的长度相等。

输入格式

一行由小写英文字母和空格组成的字符串。

字符串长度不超过1000，至少包含一个单词。

输出格式

一行和原字符串长度相等的字符串。

样例输入

```
  this   is  a sentence
```

样例输出

```
this   is   a   sentence
```

二、解析

易错点

W=1必须先处理，避免除零；读输入不能丢行首空格；输出尾部空格是答案的一部分；扫描末尾单词时先检查i<length再访问text[i]。

复杂度

O(L)时间；单词字符总长不超过L，额外存储按题目上限配置。

思考

能否原地扫描实现目的

三、参考代码（C++11）

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string text, words[1000];
    getline(cin, text);
    int spaces = 0, count = 0;
    int i = 0, length = static_cast<int>(text.size());
    while (i < length) {
        if (text[i] == ' ') {
            ++spaces;
            ++i;
        } else {
            string word;
            while (i < length && text[i] != ' ') {
                word += text[i];
                ++i;
            }
            words[count++] = word;
        }
    }
    if (count == 1) {
        cout << words[0] << string(spaces, ' ') << '\n';
        return 0;  // 单个单词没有间隔，不能除以 count - 1
    }
    int gap = spaces / (count - 1), rest = spaces % (count - 1);
    for (int k = 0; k < count; ++k) {
        if (k > 0) cout << string(gap, ' ');
        cout << words[k];
    }
    cout << string(rest, ' ') << '\n';
    return 0;
}
```

11　流感

一、题目

学校里一共有n个学生。这n个学生里一共有m对朋友关系。 在流感发作期，每个健康学生都要看望当天他生病的朋友（如果有的话），并在第二天被传染上疾病（除非他在免疫期内）； 每个生病的学生在第二天都会痊愈，并在这一天具有免疫性。从第三天起，看望生病的朋友将再次使他染上流感。 初始时（第一天），只有一个学生患有流感。试问多少天后流感会自动结束。

输入格式

第一行输入两个正整数n和m。 接下来m行每行两个正整数x,y，表示编号为x的学生和编号为y的学生是一对朋友。输入数据保证每一对朋友关系只描述一次。 最后一行输入一个正整数，代表初始时患有流感的学生的编号。

n,m <= 100,000。

输出格式

输出多少天后流感会结束。

如果天数超过2,000,000,000，则视为流感不会结束，输出-1。

样例输入

```
4 4
1 2
2 3
3 4
2 4
1
```

样例输出

```
3
```

原题提示

第一天1号学生生病，2号学生访问他； 第二天2号学生生病，其它三个学生访问他，由于1号处于免疫期，未患流感； 第三天3、4号学生生病，2号学生访问他们。 第四天3、4号学生痊愈，流感结束。

二、解析

思路：按最短距离分层传播

把学生看作点，朋友关系看作无向边。设学生到初始病人的最少朋友关系数为 d，则他在第 d+1 天第一次生病。用数组模拟队列做广度优先搜索（BFS），记录每个人的发病日，答案是所有可达学生的最大发病日。

这是图与队列的拓展题。n、m 可达 100000，不能用 n×n 的邻接矩阵。代码用 head、to、nxt 三个数组保存朋友关系：head[u] 指向 u 的第一条边，再通过 nxt 找其余边；每对朋友存两个方向。

为什么不会反复感染

任意一对朋友到起点的最短距离最多相差 1。因此，当第 d 层在第 d+1 天发病时：第 d−1 层当天正在免疫，第 d 层当天已经生病，只有第 d+1 层可能在第二天被传染。传播只会继续向外一层，不能传回之前的层。归纳可知每个可达学生恰好生病一次，其他连通分量不受影响。

因此即使朋友关系存在环，流感仍会结束；本题"只有一个初始病人、痊愈当天免疫"的条件下答案不超过 n，不会触发 -1。不能把"有环"直接判成永不结束。

样例与天数约定

样例各层依次为 {1}、{2}、{3,4}，最后一批学生在第 3 天发病，第 4 天全部健康，按题目样例输出 3。也就是输出从第 1 天开始到全部健康所经过的天数，数值等于最大发病日，不要再加 1。初始病人没有朋友时输出 1。

易错点与复杂度

不能边扫描边把"明天感染的人"当作今天的传染源；BFS 在入队时就标记，避免重复入队。关系必须双向存储，只统计初始病人所在连通分量。

时间 O(n+m)，空间 O(n+m)。

三、参考代码（C++11）

```cpp
#include <iostream>
using namespace std;

// 每条朋友关系存两个方向；大数组放在全局，避免占用过多栈空间
int head[100005], to[200005], nxt[200005];
int day[100005], que[100005];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) head[i] = -1;
    int edges = 0;
    for (int i = 0; i < m; ++i) {
        int x, y;
        cin >> x >> y;
        // head[x] 指向 x 的第一条边，nxt 串起其余边
        to[edges] = y; nxt[edges] = head[x]; head[x] = edges++;
        to[edges] = x; nxt[edges] = head[y]; head[y] = edges++;
    }
    int start;
    cin >> start;
    int front = 0, back = 0, answer = 1;
    que[back++] = start;
    day[start] = 1; // 第一日发病；0 表示尚未到达
    while (front < back) {
        int u = que[front++];
```

```
            for (int e = head[u]; e != -1; e = nxt[e]) {
                int v = to[e];
                if (day[v] != 0) continue;
                day[v] = day[u] + 1;
                if (day[v] > answer) answer = day[v];
                que[back++] = v; // 标记后入队，每人只入队一次
            }
        }
        cout << answer << '\n';
        return 0;
    }
```

12　柱状图上的最大矩形

一、题目

给定 n 个非负整数，代表柱状图上每个柱的高度（每个柱的宽度均为 1），求这个柱状图中最大的矩形面积。例如对于输入"2 1 5 6 2 3"，最大面积为10（如下图）。

输入格式

第一行是一个整数 n，代表有多少个柱形，n 小于等于 30000 第二行有 n 个整数，依次为每个柱形的高度

输出格式

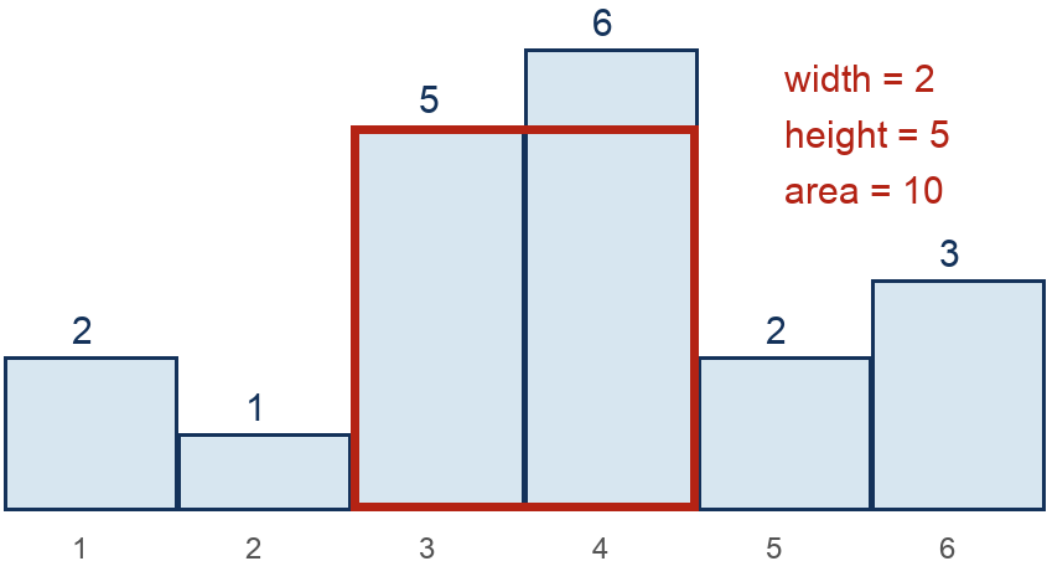
一个数字，即这个柱状图中最大的矩形面积，面积最大不超过 int 的表示范围

样例输入

```
6
2 1 5 6 2 3
```

样例输出

```
10
```



二、解析

n≤30000。朴素枚举左右端点并维护区间最小高度是O(n²)，不作为本题解法。对任何区间，最大可取矩形高度是区间最小柱高，所以目标是为各高度寻找足够宽的区间。

用st保存柱子下标，栈内高度非递减。扫描i时，如果当前柱比栈顶矮，则栈顶高度无法向右延伸到i。弹出后，右端为i−1、左端为新栈顶下标+1，宽度i−st[top]−1。用height*width更新答案。st[0]=0是左边界哨兵；末尾额外的0高度负责清算尚未弹出的正高度。

等高柱保留，因此新栈顶可能与被弹柱等高，当前计算的区间不一定是这一高度最宽的区间；更靠左的同高柱稍后弹出时会得到更宽候选，最大答案不会漏。

正确性：每个被计算的区间内所有柱都不低于弹出高度，因此是合法矩形。对任意最优区间，取其中最左的最低柱；在它右侧首次遇到更低柱时，同高右邻柱已被弹出，它会得到至少覆盖该区间的候选，故不会漏掉最优面积。每根柱入栈、出栈各至多一次，总时间O(n)，空间O(n)。虽然题目保证面积不超过int，乘法仍用long long稳妥。

样例2,1,5,6,2,3：扫描到位置5的高度2，先弹高度6，宽1面积6；再弹高度5，左边界为位置2，宽5−2−1=2，面积10。

三、参考代码（C++11）

```cpp
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    long long h[30002] = {};
    int st[30002], top = 0;
    for (int i = 1; i <= n; ++i) cin >> h[i];
    st[0] = 0; // 下标0是左边界哨兵，不弹出
    long long answer = 0;
```

```
    for (int i = 1; i <= n + 1; ++i) { // h[n+1]=0，收尾时清算正高度
        while (top > 0 && h[st[top]] > h[i]) {
            long long height = h[st[top--]];
            int width = i - st[top] - 1;
            // 弹出后，st[top]+1 到 i-1 是该高度当前可扩展的区间
            long long area = height * width;
            if (area > answer) answer = area;
        }
        st[++top] = i; // 数组模拟下标栈，高度保持非递减
    }
    cout << answer << '\n';
    return 0;
}
```

13　照亮房间

一、题目

你需要放置一盏灯来照亮房间的每个角落，问这是否能办到?

输入格式

输入包含多组数据。 每组数据第一行是正整数n(4<=n<=100)，表示房间轮廓所形成的多边形的顶点个数。 接下来n行，每行表示房间的一个顶点。 顶点按顺时针的顺序给出，房间轮廓多边形的所有边都平行于坐标轴。 输入以n=0表示结束。

输出格式

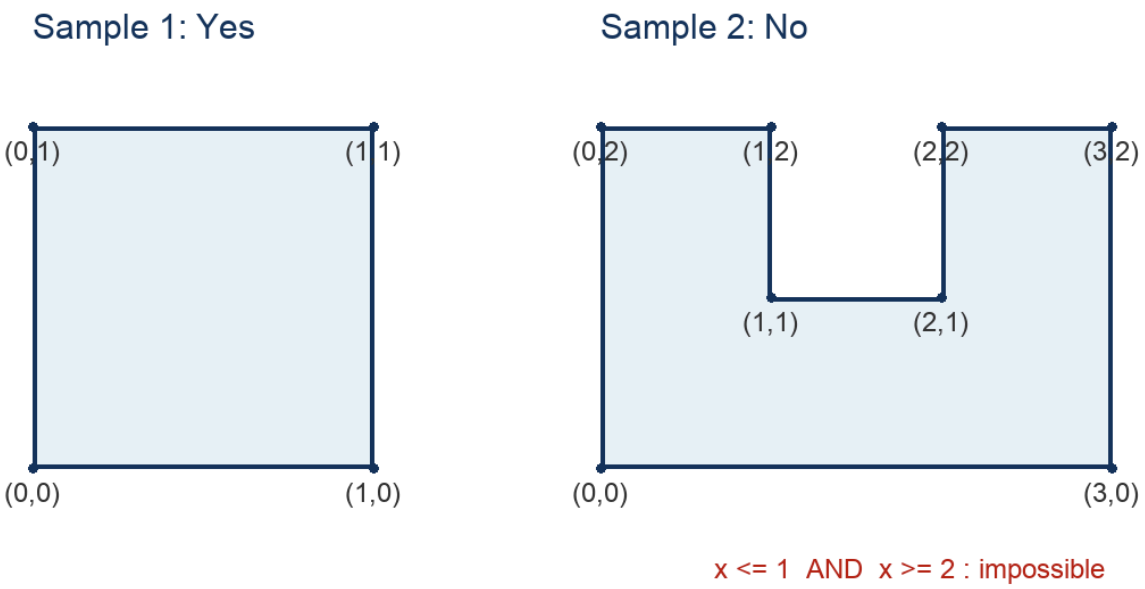
对每组数据输出一行。 如果能有一种放置方法照亮房间的所有地方，输出"Yes"，否则输出"No"

样例输入

```
4
0 0
0 1
1 1
1 0
8
0 0
0 2
1 2
1 1
2 1
2 2
3 2
3 0
0
```

样例输出

```
Yes
No
```



二、解析

正确性依据：可视核半平面交定理；正交条件将一般半平面交简化为四个上下界。时间O(n)，存顶点O(n)。可参考：多边形可视核的半平面交描述，这里不展开通用计算几何算法。

三、参考代码（C++11）

```cpp
#include <iostream>
#include <limits>
using namespace std;
int main() {
    int n;
    while (cin >> n && n != 0) {
        double x[100], y[100];
        for (int i = 0; i < n; ++i) cin >> x[i] >> y[i];
        double left = -numeric_limits<double>::infinity();
        double right = numeric_limits<double>::infinity();
        double bottom = -numeric_limits<double>::infinity();
        double upper = numeric_limits<double>::infinity();
        for (int i = 0; i < n; ++i) {
            int j = (i + 1) % n; // 最后一个顶点也要连回第一个顶点
            // 顶点顺时针给出，内部在有向边右侧
            if (x[j] > x[i]) { // 向右走：灯在边的下方
                if (y[i] < upper) upper = y[i];
            } else if (x[j] < x[i]) { // 向左走：灯在边的上方
                if (y[i] > bottom) bottom = y[i];
            } else if (y[j] > y[i]) { // 向上走：灯在边的右方
                if (x[i] > left) left = x[i];
```

```
            } else if (y[j] < y[i]) { // 向下走：灯在边的左方
                if (x[i] < right) right = x[i];
            }
        }
        // 按通常的几何题约定允许灯在边界：交集退化为线或点也非空
        cout << (left <= right && bottom <= upper ? "Yes" : "No") << '\n';
    }
    return 0;
}
```

14　炸鸡排

一、题目

小P买了n块鸡排，想将它们做成美味的炸鸡排，其中第i块鸡排需要t[i]秒炸熟。小P只有一个炸锅，炸锅内可以放置k(k≤n)块鸡排。小P是个完美主义者，他要求任意时刻炸锅内必须恰有k块鸡排。他可以在任意时刻改变锅内正在炸的鸡排，只需保证已经熟了的鸡排不能继续留在锅中。小P希望知道炸鸡排最多可以持续多少时间。

例如，小P的三块鸡排需要分别需要1,1,1的时间炸熟，炸锅内需要放置2块鸡排，那么他决定在第一个0.5秒炸1和2两块鸡排，第二个0.5秒炸2和3两块鸡排，第三个0.5秒炸1和3两块鸡排，共持续了1.5秒。

输入格式

第一行输入两个正整数n和k，k≤n 第二行输入n个正整数，代表n块鸡排分别需要炸熟的时间t[1],t[2]...t[n] 输入数据保证，n≤1000，0＜t[i]≤1000000

输出格式

输出一个双精度浮点数，代表炸鸡排最多可以持续的时间，结果保留三位小数。

样例输入

```
4 2
5 1 1 2
```

样例输出

```
4.000
```

原题提示

样例说明： 第1秒，放置1和2 第2秒，放置1和3 第3、4秒，放置1和4 至此，2，3，4已经全部熟了，无法继续进行

二、解析

定位：贪心推理拓展。允许任意时刻切换，同一块不能同时占多个位置；不要求最终所有鸡排都熟，也不能把时间限制为整数秒。

若持续T秒，需要kT的总在锅时间，每块最多贡献min(t[i],T)，故必须满足Σmin(t[i],T)≥kT。在允许暂停和任意切换的条件下，这也是可行条件。可将所选时长不超过T、总和为kT的工作量分配到k条长度T的时间轴：按顺序填满时间轴，跨界任务的两段放在相邻轴的尾部与头部，由于单任务不超过T，两段不在同一时刻重叠。这是充分性的构造解释，不需在程序里实际排出日程。

因此不能无条件回答sum/k。样例5,1,1,2与k=2：总量9看似支持4.5秒，但最长那块在4.5秒内最多贡献4.5秒，其他三块合计只有4秒，填不满两个位置。

降序排序，令剩余总时间S、剩余位置数c。若最长时间a>S/c，则任何可行时长均不超过S/c<a，这块可以全程占一个位置，将问题化成S−a和c−1。这个化简也可由可行条件解释：在T<a的范围内，min(a,T)=T，把等式两边各减去T即可。继续剥离，直到剩余最长时间≤S/c，此时所有剩余块都不超过T=S/c，且总量恰好cT，所以上界能够达到。

排序O(n²)，剥离O(n)，空间O(n)，n≤1000适用。

可参考：可抢占同速并行机调度的最优工期性质。这里只借用“可暂停、单任务不并行”的可行排程论证。

三、参考代码（C++11）

```cpp
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    int n, k, t[1000];
    cin >> n >> k;
    long long sum = 0;
    for (int i = 0; i < n; ++i) { cin >> t[i]; sum += t[i]; }
    // 基础冒泡降序排序，不使用自定义比较函数
    for (int end = n - 1; end > 0; --end)
        for (int j = 0; j < end; ++j)
            if (t[j] < t[j + 1]) {
                int temp = t[j]; t[j] = t[j + 1]; t[j + 1] = temp;
            }
    int slots = k;
    for (int i = 0; i < n; ++i) {
        // t[i] > sum/slots：这块太长，可让它全程占一个位置
        // 用整数乘法判断，避免比较时的浮点误差
        if (1LL * t[i] * slots > sum) {
            sum -= t[i];
            --slots;
        } else break;
    }
    // slots不会变为0：slots=1时单块时间不可能大于剩余总时间
    cout << fixed << setprecision(3) << double(sum) / slots << '\n';
    return 0;
}
```

15　愉悦的旋律

一、题目

在去年的计概期末，大家知道了P大富哥李哥，而李哥有个好朋友陈哥。

与李哥不同，陈哥喜欢钻研音乐，他创造了一种特殊的音乐，这种音乐可以包含 M种 不同的音符 (1<=M<=10000)，因此，每个音符都可以唯一编码为1到10000之间的一个整数。陈哥创作的每首乐曲都可以看做由 N个 (1<=N<=100000)这种音符所组成的音符序列。

例如，可以利用3种音符组成如下的音符序列：1,2,2,3,2,3,3,1

根据陈哥敏感的音乐细胞，他发现，针对每首给定的音符序列，有些子序列（不一定连续）会出现在该序列中，而有些子序列则不会出现在该序列中。

例如，对于上面例子中的音符序列，子序列"1,2,2,1"就出现在该序列中，而子序列“2,1,3”就没有出现在该序列中。

热心的陈哥告诉你：正如子序列"1,2,2,1"一样，子序列中的数不需要在原始音符序列中“连续出现”，只要其遵循原本在音符序列中的先后次序，即使子序列的各个数之间穿插有其他数字，也可认为这个子序列出现于音符序列中。

现在，给定一首已经创作完成的包含 N个 音符的音符序列，陈哥想用 M种 不同的音符构造一个子序列，使之“不出现”在给定的乐曲序列中。李哥想将其称为“崭新的旋律”，但陈哥还是喜欢称为“愉悦的旋律”，请问这个“愉悦的旋律”的最短长度是多少?

输入格式

第1行输入两个整数 N 和 M ，接下来 N 行输入音符序列。

输出格式

输出一行，包含一个整数，代表最短的“愉悦的旋律”的长度。

样例输入

```
14 5
1
5
3
2
5
1
3
4
4
2
5
1
```

```
2
3
```

样例输出

```
3
```

原题提示

样例解释：

所有长度为1和2的可能的子序列都出现了，但长度为3的子序列"2,2,4"却没有出现。

注意：你只需要告诉陈哥最短的“愉悦的旋律”的长度就行。

二、解析

思路：每凑齐全部音符，就结束一段

从左到右扫描，用数组记录当前段已经见过哪些音符。每当累计见到全部 M 种音符，就立刻结束这一段并开始下一段。设一共得到 k 个完整段，则答案为 k+1；末尾不完整的部分不计入 k。

标记数组用“轮次”代替反复清零：seen[x] 等于当前 round 表示本段见过 x。完成一段后 round 加 1、kinds 清零即可。音符的编码上界为 10000，不需要假设所有编码恰好是 1 到 M。

为什么答案恰好是 k+1

下界：长度不超过 k 的旋律都能出现。 每个完整段都含全部 M 种音符。给定任意长度 k 的旋律，第 1 个音符在第 1 段选，第 2 个在第 2 段选，依此类推，选出的位置一定递增。更短的旋律同理，所以最短缺失长度至少是 k+1。

上界：一定存在长度 k+1 的缺失旋律。 记各完整段最后读入的音符依次为 c_1、…、c_k；因为段在首次凑齐时立即结束，c_i 在该段此前从未出现。匹配 c_1 时，最早也只能到第 1 段末尾；随后匹配 c_2，最早到第 2 段末尾；依此类推，匹配 c_k 后已经越过第 k 段。尾段没有凑齐全部音符，选一个其中缺少的音符 x，则 $c_1,\dots,c_k,x$ 无法出现。推迟前面的匹配只会留下更短的后缀，也不可能成功。

样例与边界

样例第一段为 1,5,3,2,5,1,3,4，第二段为 4,2,5,1,2,3，两段都包含全部 5 种音符，所以答案为 3。按上述证明还可构造缺失旋律 4,3,1。

若原序列缺少某一种音符，则 k=0、答案为 1；若 M=1，则每个位置都是一个完整段，答案为 N+1。注意题目问的是不要求连续的子序列，不能当成连续子串处理。

复杂度

时间 O(N+10000)（含标记数组初始化），空间 O(10000)。边读边处理，不必存下整个音符序列；无需动态规划。

三、参考代码（C++11）

```cpp
#include <iostream>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    int seen[10001] = {}; // 音符编码在 1 到 10000 之间
    int round = 1, kinds = 0, blocks = 0;
    for (int i = 0; i < n; ++i) {
        int x;
        cin >> x;
        if (seen[x] != round) {
            seen[x] = round;
            ++kinds;
        }
        if (kinds == m) { // 尽早结束一个包含全部 M 种音符的段
            ++blocks;
            ++round;      // 换轮次，相当于清空本段的出现标记
            kinds = 0;
        }
    }
    cout << blocks + 1 << '\n';
    return 0;
}
```

16　啤酒厂选址

一、题目

海上有一个岛，在环海边上建有一条环岛高速公路，沿着公路有n（5 < n < 10000）个居民点，假设每个居民点有一个编号，从0开始，按顺时针依次从小到大（即，0,1，...，n-1）编号。在岛上啤酒很受青睐。某啤酒企业计划在岛上投资建一个啤酒厂，并根据啤酒需求每天向居住点送啤酒。已知两个相邻的居民点的距离以及每个居住点每天的啤酒需求量（假设每个居住点每天不超过2000桶）。假定每单位长度的路程送一桶啤酒需要的费用恒定（为单位费用）。请问，选择哪一个居民点建啤酒厂，才能使每天送啤酒的费用最小（空车不计费用）。

输入格式

第一行：为居民点数目n 后面为n行，每行为一个居民点的啤酒需求量以及按顺时针离下一个居民点的距离（均为整数,空格间隔），从编号为0的开始，按单增顺次给出。

注意：后面第n行对应于居民点（n-1)的啤酒需求量以及到编号为0的居民点距离。

输出格式

啤酒厂所在的居民点编号以及每天的运输费用，其间以逗号间隔

样例输入

```
6
500 10
300 30
350 25
400 60
700 28
200 35
```

样例输出

```
0,94100
```

二、解析

思路：枚举建厂点，计算需求加权的环上最短路

设 pos[0]=0，pos[i] 为从 0 号点顺时针走到 i 号点的累计路程。把输入的全部 n 段距离累加，得到周长 L=pos[n]；最后一段是 n−1 号点回到 0 号点，不能遗漏。

对候选建厂点 i 和需求点 j，令 d=|pos[j]−pos[i]|，两条环路的长度分别为 d 和 L−d，因此最短运输距离为 min(d,L−d)。该点的费用为 demand[j]×min(d,L−d)，对所有居民点求和，选总费用最小的建厂点。

为什么正确

环上从 i 到 j 的两条简单路径正是顺时针和逆时针两条路。空车不计费用，且题目没有车辆容量或共享路线等限制，所以每个需求点都可以独立选较短的一条路，乘以桶数后相加。再枚举所有合法建厂位置，便覆盖了全部候选方案。

样例展开

样例周长为 10+30+25+60+28+35=188。在 0 号点建厂时，各点最短距离为 0、10、40、65、63、35，总费用为：

```
500×0 + 300×10 + 350×40 + 400×65 + 700×63 + 200×35
= 94100
```

比较其他建厂点后，最优输出为 `0,94100`。

易错点与复杂度

必须按需求量加权，不能只求距离和；不能只走顺时针；输出用英文逗号。累计距离、乘积与总费用都用 long long，避免通常规模下的 int 溢出。题面没有给路段长度上界，此实现按总距离与总费用可由 64 位整数表示处理。

题面没有规定最优位置并列时的选择，本实现按编号递增枚举且仅在费用严格减小时更新，保留编号最小的最优位置。

时间 $O(n^2)$，空间 $O(n)$。$n<10000$，最多接近一亿次简单费用计算；这是便于基础课堂讲解的枚举版。原题文件没有给出时间限制，不能仅据此保证在线评测时限；若时限严格，可进一步用前缀和与双指针优化。

三、参考代码（C++11）

```cpp
#include <iostream>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    long long demand[10000], pos[10001] = {};
    for (int i = 0; i < n; ++i) {
        long long length;
        cin >> demand[i] >> length;
        pos[i + 1] = pos[i] + length;
    }
    long long circumference = pos[n]; // 包括末点到 0 号点的路段
    long long best = -1;
    int location = 0;
    for (int i = 0; i < n; ++i) {
        long long cost = 0;
        for (int j = 0; j < n; ++j) {
            long long distance = pos[j] - pos[i];
            if (distance < 0) distance = -distance;
            // 环上两条路分别长 distance 和 circumference-distance
            if (circumference - distance < distance)
                distance = circumference - distance;
            cost += demand[j] * distance;
        }
        // 只在严格更小时更新：并列时保留编号最小的点
        if (best == -1 || cost < best) {
            best = cost;
            location = i;
        }
    }
    cout << location << ',' << best << '\n';
    return 0;
}
```